



كلية تكنولوجيا الصناعة والطاقة

Information Technology Department



جامعة برج العرب للتكنولوجيه

BORG AL ARAB TECHNOLOGICAL UNIVERSITY

PROGRAMMING IN C

First Year –Information Technology Program

Spring 2026



LECTURE 12

PROBLEM SOLVING & PROGRAM DESIGN

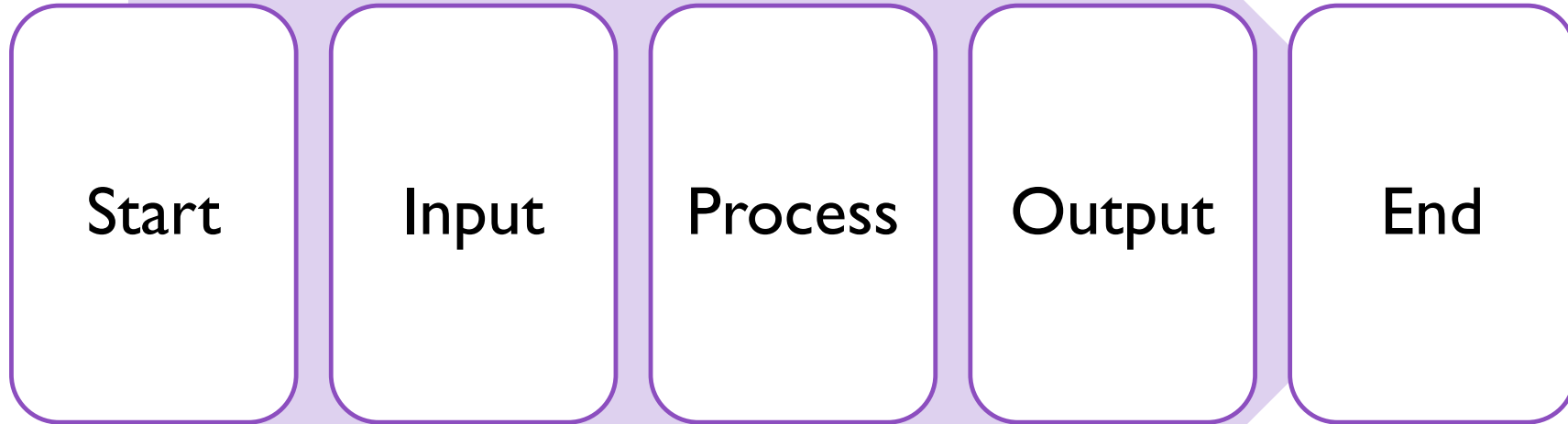
LEARNING OBJECTIVES

- Break problems
 - Think logically
 - Design algorithms
 - Convert to C
 - Debug
-

PROGRAMMING REALITY

- Programming = Thinking
 - Not just writing code
-

FLOW CHART THINKING



PROBLEM SOLVING STEPS:

- Understand the problem
 - Analyze inputs & outputs
 - Break into small steps
 - Write algorithm (logic)
 - Convert to C code
 - Test and debug
-

UNDERSTAND THE PROBLEM

Understand the Problem

Example: “Calculate sum of two numbers”

STEP 2 – IDENTIFY INPUT & OUTPUT

Ask:

- What is given? Input → two numbers
- What is required? Output → sum

Input	Process	Output
a, b	$\text{sum} = a + b$	sum

STEP 3 – BREAK THE PROBLEM

Step 3 – Break the Problem

Example: To calculate sum:

- Start
- Read a, b
- Compute sum
- Print result
- End

Small steps = easier code

STEP 4 – WRITE ALGORITHM

Step 4 – Write Algorithm

- 1. Start
- 2. Input a, b
- 3. $\text{sum} = a + b$
- 4. Print sum
- 5. End

STEP 5 – CONVERT TO C CODE

```
#include <stdio.h>
```

```
int main() {  
    int a, b, sum;  
    scanf("%d %d", &a, &b);  
    sum = a + b;  
    printf("%d", sum);  
    return 0;  
}
```

STEP 6 – TEST THE PROGRAM

- Try different inputs
- Check correctness
- Fix errors, check edges values (number unexpected from users, maybe they enter characters not words!)

DECISION EXAMPLE

- **Problem:** input two numbers, then print the largest
- Thinking:**

Step 1 understand the problem

Compare $\rightarrow$ if $(a > b)$

Step 2 Find the inputs and outputs

Input $\rightarrow$ a, b

Output: print largest

ALGORITHM IN ANY LANGUAGE!

- 1. Start
- 2. Input a, b
- 3. If $a > b \rightarrow$ print a
- 4. Else $\rightarrow$ print b
- 5. End

C CODE

```
#include <stdio.h>

int main() {
    int a, b;
    scanf("%d %d", &a, &b);
    if (a > b)
        printf("%d ", a);
    else
        printf("%d", b);
    return 0;
}
```

THINKING EXAMPLE 2 (CONDITION)

Problem: Check if number is even or odd

Step 1 understand the problem

Process: Even means check if $\rightarrow \text{number \% 2} == 0$

Step 2 Find the inputs and outputs

Input : number

Output: print even or odd

ALGORITHM + CODE

```
#include <stdio.h>

int main() {
    int num;
    scanf("%d %d", &num);
    if (num % 2 == 0)
        printf("Even %d", num);
    else
        printf("Odd ", num);
    return 0;
}
```

LOOP THINKING

- **Problem:** Print numbers from 1 to 5

Step 1 understand the problem

Thinking:

- Start from 1
 - Stop at 5
 - Increase each time
 - **Step 2 Find the inputs and outputs**
 - **Input: i**
 - **Output: print i**
-

```
for(int i = 1; i <= 5; i++) {  
    printf("%d ", i);  
}
```

Key Thinking Patterns

ARRAYS

Problem Type

Decision

Repetition

Calculation/ Logic

Multiple values

Concept

if / else switch case

Loops for , while , do while

Operators + , - , * , % , / , > , < , == , ...

Arrays

COMMON MISTAKES

- Writing code before thinking
 - Not identifying input/output
 - Ignoring edge cases
 - Copy-pasting without understanding
-

DEBUGGING

Ask:

- What is wrong?
- Where is the error?
- What happened?
- What expected?

Fix step by step

- 
-
- **Problem:** Find sum of first N numbers

Step 1 understand the problem

from 1 to N ,Add numbers

Step 2 Find the inputs and outputs

- Input $\rightarrow$ N
- Process $\rightarrow$ loop from 1 to N ,Add numbers
- output $\rightarrow$ print sum of N numbers

CODE IN C

```
int n, sum = 0;
scanf("%d", &n);
for(int i = 1; i <= n; i++) {
    sum += i;
}
printf("%d", sum);
```

SQUARE OF A NUMBER

Problem : print the Square of a Number

Step 1: Understand the Problem

- We need to calculate the **square** of a number

Step 2: Identify Input & Output

- Input : number
- Process: $\text{Square} = \text{number} \times \text{number}$
- Output: print

STEP 3: BREAK INTO STEPS

Break into Steps

- Start
- Read number
- Calculate square
- Print result
- End

C CODE

```
int n, square;  
scanf("%d", &n);  
square = n * n;  
printf("%d", square);
```

EXERCISE : POSITIVE OR NEGATIVE

Exercise : Positive or Negative

Step 1: Understand

- Check if number is:
 - Positive
 - Negative
 - Or Zero

Step 2: Input & Output

- Input → number (n)
- Output → "Positive" / "Negative" / "Zero"

EXERCISE : POSITIVE OR NEGATIVE

Step 3: Thinking (Important!)

- Input n
- If $n > 0 \rightarrow$ Positive
- If $n < 0 \rightarrow$ Negative
- Else $\rightarrow$ Zero
- Output positive or negative

```
int n;  
scanf("%d", &n);  
if (n > 0)  
    printf("Positive");  
else if (n < 0)  
    printf("Negative");  
else  
    printf("Zero");
```

EXERCISE : MULTIPLICATION TABLE

■ Exercise : Multiplication Table

Step 1: Understand

■ Print table of a number (e.g., 5):

■ $5 \times 1 = 5$

■ $5 \times 2 = 10$

■ ...

EXERCISE : MULTIPLICATION TABLE

Step 2: Input & Output

- Input → number (n)
- Output → table from 1 to 10

Step 3: Thinking

- Use a loop
- Repeat from 1 to 10
- Multiply each time

EXERCISE : MULTIPLICATION TABLE

Step : C Code

```
int n;  
  
scanf("%d", &n);  
  
for (int i = 1; i <= n; i++) {  
    printf("%d x %d = %d\n", n, i, n * i);  
}
```

CHALLENGE EXERCISE: FACTORIAL

Challenge Exercise: Factorial

Step 1: Understand

- Factorial of n:

$$n! = n \times (n-1) \times \dots \times 1$$

CHALLENGE EXERCISE: FACTORIAL

Step 2: Input & Output

- Input $\rightarrow$ number (n)
- Output $\rightarrow$ print factorial

Step 3: Thinking

- Use loop
- Start from 1
- Multiply each number

CHALLENGE EXERCISE: FACTORIAL

```
int n, fact = 1;  
scanf("%d", &n);  
for (int i = n; i >= 1; i--) {  
    fact = fact * i;  
}  
printf("%d", fact);
```

REAL-LIFE PROBLEM: GRADING

Real-life Problem: Grading

Understand the Problem

We want to:

- Take a student's **marks**
 - print the **grade**
-

REAL-LIFE PROBLEM: GRADING

Step 1: Define the Rules

Marks

90 – 100

80 – 89

70 – 79

60 – 69

< 60

Grade

A

B

C

D

F

REAL-LIFE PROBLEM: GRADING

Step 2: Input & Output

- Input → marks
- Output → grade

Step 3: Thinking Process

- Do we need a loop? No
 - Do we need conditions? Yes
-

Step 4: Break into Steps

- Start
- Read marks
- Check range
- Print grade
- End

- **REAL-LIFE PROBLEM: GRADING**

- **Step 5: Algorithm**

1. Start
2. Input marks
3. If marks $\geq 90 \rightarrow$ Print "A"
4. Else if marks $\geq 80 \rightarrow$ Print "B"
5. Else if marks $\geq 70 \rightarrow$ Print "C"
6. Else if marks $\geq 60 \rightarrow$ Print "D" 7.
- Else $\rightarrow$ Print "F"
8. End

• REAL-LIFE PROBLEM: GRADING

```
#include <stdio.h>
```

```
int main() {
```

```
    int marks;
```

```
    scanf("%d", &marks);
```

```
    if (marks >= 90)
```

```
        printf("Grade A");
```

```
    else if (marks >= 80)
```

```
        printf("Grade B");
```

```
    else if (marks >= 70)
```

```
        printf("Grade C");
```

```
    else if (marks >= 60)
```

```
        printf("Grade D");
```

```
    else
```

```
        printf("Grade F");
```

```
    return 0;
```

```
}
```

■ Step 9: Edge Cases

Now Think:

- What if marks = 100?
- What if marks = 0?
- What if marks = -5 or 150?

IMPROVED VERSION

```
if (marks < 0 || marks > 100)
```

```
    printf("Invalid input");
```

```
else if (marks >= 90)
```

```
    printf("A");
```

```
else if (marks >= 80)
```

```
    printf("B");
```

```
else if (marks >= 70)
```

```
    printf("C");
```

```
else if (marks >= 60)
```

```
    printf("D");
```

```
else
```

```
    printf("F");
```

GOLDEN RULES

- Think first
 - Break steps
 - Practice
-

SUMMARY

- Programming = Logic + Practice
-



■ **Happy Coding**